

TuniTech Abroad - Service Contracts v1

1. Document Purpose

This document defines the internal service contracts for TuniTech Abroad MVP.

It defines how the main business services should behave before implementation starts.

The goal is to prevent business logic from being scattered across Django views, Celery tasks, Django models, templates, and admin actions.

Core rule:

```
Views call services.  
Celery tasks call services.  
Models store data.  
Services own business logic.
```

This document is not source code, but it should map directly into Python service classes/functions.

2. Service Architecture Principle

TuniTech Abroad should be implemented as a modular Django monolith.

Each Django app owns its own service layer:

```
apps/  
  jobs/services/  
  cvs/services/  
  skills/services/  
  matching/services/  
  recommendations/services/  
  llm/services/  
  notifications/services/  
  privacy/services/  
  analytics/services/
```

Business logic should not live directly inside views.

Bad:

```
def job_detail_view(request, public_id):  
    # fetch job  
    # parse user skills  
    # calculate match  
    # save match  
    # call LLM  
    # render template
```

Good:

```
def job_detail_view(request, public_id):  
    job = JobQueryService.get_public_job(public_id)  
    return render(request, "jobs/detail.html", {"job": job})
```

For match calculation:

```
result = MatchScoringService.calculate_for_user_job(user=user, job=job)
```

3. Shared Contract Rules

3.1 Service Input Rules

Services should accept model instances, IDs/public IDs, validated DTOs, and plain Python values.

Services should not accept raw HTTP request objects, unvalidated form data, template context dictionaries, or raw external API payloads outside ingestion/normalization services.

3.2 Service Output Rules

Services should return model instances, typed result objects, dictionaries with stable keys, or status/result classes.

Services should not return rendered HTML, raw HTTP responses, or template-specific context unless the service is explicitly a presentation/helper service.

3.3 Error Handling Rules

Services should raise domain-specific exceptions or return structured failure results.

Recommended base exceptions:

```
ServiceError
ValidationServiceError
ExternalAPIServiceError
LLMServiceError
PermissionServiceError
NotFoundServiceError
RateLimitServiceError
```

Celery tasks should catch service exceptions and update status models.

Views should convert service exceptions into user-friendly messages.

3.4 Transaction Rules

Services that create or update multiple related records should use database transactions.

Examples:

```
CV upload metadata + parse status update
RawJobRecord update + NormalizedJob update
Recommendation refresh replacing old recommendations
Account deletion
Email batch creation + email events
```

3.5 Idempotency Rules

Services called by Celery tasks must be safe to retry.

Examples:

```
Normalizing same RawJobRecord twice should not duplicate NormalizedJob.
Sending same weekly digest twice should be prevented by idempotency_key.
Refreshing recommendations twice should update or replace existing records safely.
```

3.6 Logging Rules

Services should log operational events, but must not log full CV text, full raw personal profile, secrets, OAuth tokens, OpenRouter API key, or France Travail credentials.

Log IDs, statuses, counts, and error summaries.

4. Shared DTOs and Result Shapes

These DTOs can be implemented as dataclasses, Pydantic models, or plain dictionaries. Simple dataclasses are enough for MVP.

4.1 ServiceResult

```
@dataclass
class ServiceResult:
    success: bool
    message: str | None = None
    data: dict | None = None
    error_code: str | None = None
```

4.2 PaginatedResult

```
@dataclass
class PaginatedResult:
    items: list
    total_count: int
    page: int
    page_size: int
    has_next: bool
    has_previous: bool
```

4.3 SkillExtractionResult

```
@dataclass
class SkillExtractionResult:
    canonical_skills: list
    unmatched_candidates: list
    raw_candidates: list[str]
    confidence: float | None = None
```

4.4 FitScoreResult

```
@dataclass
class FitScoreResult:
    fit_score: int
    technical_skills_score: int
    experience_score: int
    role_title_score: int
    language_score: int
    location_score: int
    strong_skills: list[str]
    missing_required_skills: list[str]
    missing_optional_skills: list[str]
    risk_flags: list[str]
    profile_signals: list[str]
    recommended_actions: list[str]
```

4.5 RecommendationResult

```
@dataclass
class RecommendationResult:
    recommendations_created: int
    recommendations_updated: int
    scored_jobs_count: int
    candidate_jobs_count: int
    run_id: int | None = None
```

4.6 LLMRunResult

```
@dataclass
```

```
class LLMSuccessResult:
    success: bool
    cache_hit: bool
    response_json: dict | None
    response_text: str | None
    usage_log_id: int | None
    error_message: str | None = None
```

5. Accounts and Profile Services

5.1 AccountProvisioningService

Location:

```
apps/accounts/services/account_provisioning.py
```

Purpose: handles post-signup user setup after email/password registration, Google OAuth signup, or GitHub OAuth signup.

Owns: creating CandidateProfile, creating EmailPreference, recording signup event, pre-filling profile from auth provider where possible.

Does not own: password validation, OAuth handshake, email verification mechanics. Those belong to django-allauth.

Contract:

```
AccountProvisioningService.provision_new_user(user: User, provider: str | None = None) ->
CandidateProfile
```

Side effects:

```
CandidateProfile created if missing
EmailPreference created if missing
UserEvent(signup_completed)
```

Idempotency: calling this multiple times must not create duplicate profile or preferences.

Errors: ValidationServiceError.

5.2 ProfileCompletenessService

Location:

```
apps/profiles/services/completeness.py
```

Purpose: calculates profile completeness score and status.

Contract:

```
ProfileCompletenessService.calculate(profile: CandidateProfile) -> dict
```

Output:

```
{
    "score": 0-100,
    "status": "incomplete" | "usable" | "strong",
    "missing_fields": [...]
}
```

Weighting:

```
Active CV uploaded: 20
```

Main skills: 20
Target role/type: 15
Experience level/years: 15
French/English level: 10
Contact/profile links: 10
Location/relocation/remote preference: 10

Status rules:

Below 50 = incomplete
50-79 = usable
80+ = strong

A separate method can persist the result:

```
ProfileCompletenessService.update_profile_score(profile) -> CandidateProfile
```

5.3 ProfileUpdateService

Location:

```
apps/profiles/services/profile_update.py
```

Purpose: handles user profile updates and triggers dependent actions.

Contract:

```
ProfileUpdateService.update_profile(user: User, cleaned_data: dict) -> CandidateProfile
```

Owns: updating CandidateProfile, updating ProfileSkill if skills are edited, recalculating completeness, marking recommendations stale, recording profile events.

Does not own: rendering forms, validating HTML forms, calculating new recommendations synchronously.

Side effects:

CandidateProfile updated
ProfileSkill rows updated
JobRecommendation rows marked stale
Recommendation refresh task may be enqueued

Errors: PermissionServiceError, ValidationServiceError.

6. Skill Services

6.1 SkillNormalizerService

Location:

```
apps/skills/services/normalizer.py
```

Purpose: maps raw skill strings to canonical Skill records. This is one of the most important services in the product.

Contract:

```
SkillNormalizerService.normalize_many(  
    raw_skills: list[str],  
    source_type: str,  
    source_id: int | None = None  
) -> SkillExtractionResult
```

source_type values:

cv
job
quick_match
manual

Processing rules:

1. Lowercase
2. Strip punctuation
3. Normalize accents
4. Collapse whitespace
5. Exact SkillAlias.normalized_alias lookup
6. Optional fuzzy lookup later
7. Return canonical Skill if matched
8. Create/update UnmatchedSkillCandidate if not matched

Side effects: may create/update UnmatchedSkillCandidate. Must not auto-create new Skill from unknown text.

Idempotency: same unmatched skill should increment occurrence_count, not create duplicates.

6.2 SkillSeedService

Location:

apps/skills/services/seed.py

Purpose: seeds initial canonical skills and aliases.

Contract:

```
SkillSeedService.seed_initial_taxonomy() -> ServiceResult
```

Side effects: creates Skill and SkillAlias rows.

Idempotency: safe to run multiple times using get_or_create/update_or_create.

MVP seed size: 200-300 canonical skills and 500-1000 aliases.

6.3 UnmatchedSkillReviewService

Location:

apps/skills/services/review.py

Purpose: supports admin review of unmatched skill candidates.

Contracts:

```
UnmatchedSkillReviewService.map_candidate(candidate_id: int, skill_id: int, reviewed_by: User) -> UnmatchedSkillCandidate
UnmatchedSkillReviewService.ignore_candidate(candidate_id: int, reviewed_by: User) -> UnmatchedSkillCandidate
```

Side effects: marks candidate as mapped or ignored, optionally creates SkillAlias from normalized text, updates reviewed_by/reviewed_at.

Permission: staff/admin only.

7. Job Source and Ingestion Services

7.1 FranceTravailClient

Location:

```
apps/jobs/services/france_travail/client.py
```

Purpose: low-level client for France Travail API. This service owns external HTTP calls.

Owns: authentication/token retrieval, token refresh, API search requests, pagination/range handling, request retry/backoff, API error normalization.

Does not own: database writes, normalization, skill extraction, recommendation refresh.

Contracts:

```
FranceTravailClient.search_offers(params: dict) -> dict
FranceTravailClient.get_offer_detail(source_job_id: str) -> dict
```

Output: raw API response dictionary.

Errors: ExternalAPIServiceError, RateLimitServiceError.

Retry on timeout, temporary 5xx, network failure, token expiry after refresh. Do not retry endlessly.

7.2 JobIngestionService

Location:

```
apps/jobs/services/ingestion.py
```

Purpose: coordinates ingestion from a job source into RawJobRecord.

Contract:

```
JobIngestionService.sync_source(source_slug: str, trigger_type: str = "scheduled") ->
IngestionRun
```

MVP source slug:

```
france_travail
```

Processing flow:

1. Get active JobSource.
2. Create IngestionRun(status=running).
3. Build configured search queries.
4. Call FranceTravailClient.
5. For each raw offer:
 - compute source_job_id
 - compute payload_hash
 - create/update RawJobRecord
 - update first_seen_at/last_seen_at/last_fetched_at
 - enqueue normalization if new or changed
6. Mark stale/expired jobs.
7. Update IngestionRun counts.
8. Finish IngestionRun.

Side effects: creates/updates IngestionRun and RawJobRecord; enqueues normalization and skill extraction.

Idempotency: unique(source, source_job_id) prevents duplicates.

7.3 JobFixtureIngestionService

Location:

apps/jobs/services/fixture_ingestion.py

Purpose: allows development before real France Travail API access is fully stable.

Contract:

```
JobFixtureIngestionService.load_fixture_file(path: str, source_slug: str =
"france_travail") -> IngestionRun
```

Rule: fixture ingestion must use the same RawJobRecord and normalization pipeline as real ingestion.

7.4 JobFreshnessService

Location:

apps/jobs/services/freshness.py

Purpose: updates job freshness states.

Contract:

```
JobFreshnessService.mark_stale_and_expired(now=None) -> dict
```

Rules:

```
if expires_at exists and expires_at < now:
    status = expired
elif last_seen_at older than job_stale_after_hours:
    status = stale
elif last_seen_at older than job_removed_after_hours:
    status = removed
else:
    status = active
```

Defaults:

```
job_stale_after_hours = 24
job_removed_after_hours = 72
```

7.5 JobRevalidationService

Location:

apps/jobs/services/revalidation.py

Purpose: refreshes a single high-intent job before detail/apply action if old.

Contract:

```
JobRevalidationService.revalidate_if_needed(job: NormalizedJob) -> NormalizedJob
```

If job.last_fetched_at is older than job_revalidate_after_hours, refresh from source if supported.

Default:

```
job_revalidate_after_hours = 6
```

External API errors should not break page rendering. Return existing job with warning if refresh fails.

8. Job Normalization and Search Services

8.1 JobNormalizationService

Location:

```
apps/jobs/services/normalization.py
```

Purpose: converts RawJobRecord into NormalizedJob.

Contract:

```
JobNormalizationService.normalize(raw_record: RawJobRecord) -> NormalizedJob
```

Processing rules:

1. Read raw payload.
2. Map source fields defensively.
3. Extract title/company/location/contract/description.
4. Detect job type.
5. Detect remote type.
6. Detect experience level.
7. Detect language requirements.
8. Build source_url if available.
9. Create/update NormalizedJob.
10. Update search_vector.
11. Mark raw normalization_status.

Missing fields must not crash normalization.

8.2 JobClassificationService

Location:

```
apps/jobs/services/classification.py
```

Purpose: detects job type, remote type, experience level, and language requirements.

Contract:

```
JobClassificationService.classify(payload: dict, description: str, title: str) -> dict
```

Output:

```
{
  "job_type": "internship" | "full_time_job" | "apprenticeship" | "contract" | "unknown",
  "remote_type": "remote" | "hybrid" | "on_site" | "unknown",
  "experience_level": "internship" | "junior" | "mid_level" | "senior" | "unknown",
  "language_requirements": {
    "french": "required" | "preferred" | "unknown",
    "english": "required" | "preferred" | "unknown"
  }
}
```

Keyword examples: stage, stagiaire, PFE, fin d'études, alternance, apprentissage, télétravail, remote, hybride, présentiel, sur site.

8.3 JobSkillExtractionService

Location:

```
apps/jobs/services/skill_extraction.py
```

Purpose: extracts canonical skills from normalized jobs.

Contract:

```
JobSkillExtractionService.extract_for_job(job: NormalizedJob) -> SkillExtractionResult
```

Flow:

1. Extract candidate skill strings using rule-based methods.
2. Optionally call LLM job_skill_extraction if needed.
3. Normalize raw skills through SkillNormalizerService.
4. Create/update NormalizedJobSkill.
5. Update required_skills_json and optional_skills_json.
6. Update skill_extraction_status.
7. Rebuild search_vector.

LLM failure should not fail extraction if rule-based extraction succeeded.

8.4 JobSearchService

Location:

```
apps/jobs/services/search.py
```

Purpose: performs public/local database job search. Must not call external job APIs.

Contract:

```
JobSearchService.search(filters: dict, user: User | None = None) -> PaginatedResult
```

Filters include query, location, contract_type, job_type, remote_type, experience_level, page, page_size, and sort.

Search rules: use PostgreSQL full-text search.

Search vector weights:

```
title = A
required skills = A
optional skills = B
company = B
location = C
description = D
```

Best-match sort is only available when user is authenticated and has usable profile. Otherwise fallback to relevance/newest.

8.5 JobQueryService

Location:

```
apps/jobs/services/query.py
```

Purpose: centralized safe job retrieval.

Contract:

```
JobQueryService.get_public_job(public_id: UUID) -> NormalizedJob
```

Public pages retrieve jobs by public_id, never internal id.

9. CV Services

9.1 CVUploadService

Location:

```
apps/cvs/services/upload.py
```

Purpose: handles CV upload metadata, validation, and task enqueueing.

Contract:

```
CVUploadService.upload_cv(user: User, uploaded_file) -> CVUpload
```

Validation:

```
file is PDF
file size <= max_cv_upload_size_mb
user accepted cv_processing consent
```

Default max size: 5 MB.

Flow:

1. Validate file.
2. Compute file_hash.
3. Deactivate previous active CV.
4. Save file in private media.
5. Create CVUpload(parse_status=pending, is_active=true).
6. Enqueue parse_cv task.
7. Record UserEvent(cv_uploaded).

9.2 CVTextExtractionService

Location:

```
apps/cvs/services/text_extraction.py
```

Purpose: extracts raw text from PDF.

Contract:

```
CVTextExtractionService.extract_text(cv_upload: CVUpload) -> dict
```

Use PyMuPDF/pdfplumber. No OCR in MVP.

If text length is too low, return success=false with reason=too_little_text.

9.3 CVDeterministicExtractorService

Location:

```
apps/cvs/services/deterministic_extractor.py
```

Purpose: extracts exact fields using deterministic logic.

Contract:

```
CVDeterministicExtractorService.extract(raw_text: str) -> dict
```

Extracts email, phone, LinkedIn URL, GitHub URL, portfolio URL, and website URL.

Does not reliably extract years of experience, project structure, education hierarchy, or full work history. Those belong to LLM structured extraction.

9.4 CVLLMExtractionService

Location:

```
apps/cvs/services/llm_extraction.py
```

Purpose: uses OpenRouter to extract structured CV data from raw text.

Contract:

```
CVLLMExtractionService.extract_structured(cv_upload: CVUpload, raw_text: str) -> LLMRunResult
```

Uses PromptVersion cv_extraction:v1 and cache key cv_hash + prompt_version.

Required output schema includes candidate identity, skills, experience, projects, education, certifications, languages, estimated_years_experience, warnings, and confidence.

9.5 CVParsingService

Location:

```
apps/cvs/services/parsing.py
```

Purpose: coordinates full CV parsing.

Contract:

```
CVParsingService.parse(cv_upload: CVUpload) -> CVParsedData
```

Flow:

1. Set CVUpload.parse_status=processing.
2. Extract raw text.
3. If text unreadable, set failed/too_little_text and stop.
4. Run deterministic extraction.
5. Run LLM structured extraction if within limit.
6. Merge deterministic + LLM output.
7. Normalize extracted skills.
8. Create/update CVParsedData.
9. Create/update ProfileSkill from canonical skills.
10. Prefill CandidateProfile draft fields where empty.
11. Set parse_status=parsed or parsed_with_warnings.
12. Trigger profile completeness recalculation.

Recommendations should generally be triggered after user confirmation, not immediately.

9.6 CVDeletionService

Location:

```
apps/cvs/services/deletion.py
```

Purpose: safely deletes CV data and physical files.

Contract:

```
CVDeletionService.delete_cv(user: User, cv_public_id: UUID) -> ServiceResult
```

Rules:

```
Normal app code uses CVUpload.objects.  
Privacy/admin code may use CVUpload.all_objects.
```

Flow:

1. Verify ownership.
2. Remove physical file.
3. Soft-delete CVUpload.
4. Delete or clear CVParsedData.
5. Remove unconfirmed ProfileSkill rows sourced only from that CV.
6. Mark recommendations stale.
7. Record UserEvent.

10. Matching Services

10.1 MatchScoringService

Location:

apps/matching/services/scoring.py

Purpose: calculates deterministic fit score between profile/CV and job. This is the core scoring authority.

Contract:

```
MatchScoringService.calculate(  
    profile: CandidateProfile,  
    job: NormalizedJob,  
    cv_upload: CVUpload | None = None  
) -> FitScoreResult
```

Formula:

```
fit_score =  
technical_skills_score * 0.45  
+ experience_score * 0.20  
+ role_title_score * 0.15  
+ language_score * 0.10  
+ location_score * 0.10
```

Technical score uses ProfileSkill and NormalizedJobSkill. Required skills are 80% of technical score and optional skills are 20%.

If no required skills are detected, use all detected skills and add low_confidence_job_skills signal.

Risk flags: experience_too_low, missing_required_skills, french_level_missing, english_level_missing, profile_incomplete, job_may_be_expired, internship_type_unclear.

Profile signals: profile_signal_missing_github, profile_signal_missing_linkedin, profile_signal_missing_portfolio. Profile signals do not reduce score unless explicitly relevant.

Side effects: none. Pure calculation service.

10.2 MatchResultService

Location:

apps/matching/services/match_result.py

Purpose: creates and retrieves persisted match results.

Contract:

```
MatchResultService.create_match_result(  
    user: User,  
    job: NormalizedJob,  
    cv_upload: CVUpload | None = None  
) -> MatchResult
```

Flow:

1. Verify user has profile.
2. Verify job is visible.
3. Get active CV if not supplied.
4. Call `MatchScoringService.calculate`.
5. Store `profile_snapshot_json`.
6. Store `job_snapshot_json`.
7. Create `MatchResult`.
8. Record `UserEvent(match_generated)`.

Does not own LLM explanation generation or recommendation refresh.

10.3 QuickMatchService

Location:

`apps/matching/services/quick_match.py`

Purpose: calculates anonymous teaser match without account or CV.

Contract:

```
QuickMatchService.run_quick_match(  
    session_key: str,  
    job: NormalizedJob,  
    entered_skills: list[str],  
    experience_level: str,  
    french_level: str,  
    ip_address: str | None = None  
) -> QuickMatchSession
```

Rules:

No CV upload.
No account required.
No LLM.
Expires after 24 hours.
Rate-limited per session/IP.

10.4 MatchExplanationService

Location:

`apps/matching/services/explanation.py`

Purpose: generates optional LLM explanation for an existing `MatchResult`.

Contract:

```
MatchExplanationService.generate(match_result: MatchResult, user: User) -> MatchResult
```

LLM receives structured deterministic data only: fit score, score breakdown, strong skills, missing skills, risk flags, profile signals, job title, and job summary. It must not invent new scores.

Cache key: `cv_hash + job_id + prompt_version`.

If LLM fails, the match result remains valid.

11. Recommendation Services

11.1 RecommendationService

Location:

```
apps/recommendations/services/recommendation.py
```

Purpose: generates and stores job recommendations.

Contract:

```
RecommendationService.refresh_for_user(user: User, trigger_type: str) ->  
RecommendationResult
```

Trigger values: cv_uploaded, cv_replaced, profile_updated, new_jobs_imported, dashboard_stale_refresh, nightly_refresh, manual_admin.

Flow:

1. Create RecommendationRun(status=running).
2. Validate user has usable profile.
3. Get active CV if available.
4. Prefilter active jobs.
5. For each candidate job, call MatchScoringService.calculate.
6. Calculate ranking_score.
7. Store top recommendations.
8. Mark old recommendations stale or update existing.
9. Complete RecommendationRun.

Prefilter rules: country France, status active, job_type matches target_type where possible, skill overlap exists, role/title rough match, not dismissed.

Ranking formula:

```
ranking_score = fit_score + freshness_boost + target_type_boost + user_preference_boost -  
stale_job_penalty
```

11.2 RecommendationQueryService

Location:

```
apps/recommendations/services/query.py
```

Purpose: reads recommendations for dashboard.

Contract:

```
RecommendationQueryService.get_dashboard_recommendations(user: User, limit: int = 20) ->  
list[JobRecommendation]
```

Behavior: return fresh recommendations when available; if stale, return existing recommendations and enqueue refresh; if none, enqueue generation and return pending state.

11.3 SavedJobService

Location:

```
apps/recommendations/services/saved_jobs.py
```

Contracts:

```
SavedJobService.save_job(user: User, job_public_id: UUID) -> SavedJob  
SavedJobService.remove_saved_job(user: User, job_public_id: UUID) -> ServiceResult
```

Saving same job twice returns existing SavedJob.

11.4 ActiveUserRecommendationService

Location:

```
apps/recommendations/services/active_users.py
```

Contract:

```
ActiveUserRecommendationService.get_active_users(days: int = 14) -> QuerySet[User]
```

Active user definition: logged in within last 14 days, has usable profile, and has active CV or enough manual profile data.

Used by nightly recommendation refresh and weekly digest eligibility.

12. LLM Services

12.1 OpenRouterClient

Location:

```
apps/llm/services/openrouter_client.py
```

Purpose: low-level OpenRouter API client.

Contract:

```
OpenRouterClient.chat_json(  
    model: str,  
    messages: list[dict],  
    max_tokens: int,  
    temperature: float = 0  
) -> dict
```

Owns HTTP call, headers, API key usage, timeout, and provider error parsing.

Does not own prompt construction, cache logic, or business-specific extraction.

12.2 PromptRunnerService

Location:

```
apps/llm/services/prompt_runner.py
```

Purpose: runs versioned prompts through OpenRouter with caching and logging.

Contract:

```
PromptRunnerService.run(  
    purpose: str,  
    input_payload: dict,  
    user: User | None = None,  
    cache_key: str | None = None  
) -> LLMLRunResult
```

Flow:

1. Get active PromptVersion for purpose.
2. Build input_hash.
3. Resolve cache_key.
4. Check LLMLCacheEntry.
5. Check rate limits.
6. Call OpenRouterClient.

7. Validate JSON if expected.
8. Save LLMCacheEntry.
9. Save LLMUsageLog.
10. Return LLMRunResult.

12.3 LLMRateLimitService

Location:

```
apps/llm/services/rate_limits.py
```

Contract:

```
LLMRateLimitService.assert_allowed(user: User | None, purpose: str) -> None
```

Default MVP limits: max_llm_calls_per_user_day = 10, max_match_explanations_per_user_day = 5.

12.4 LLMOutputValidationService

Location:

```
apps/llm/services/output_validation.py
```

Contract:

```
LLMOutputValidationService.validate(purpose: str, response_json: dict) -> dict
```

For match explanation, output must not include fake score, guaranteed employment, guaranteed visa, legal advice, or invented job requirements.

13. Notification Services

13.1 EmailSenderService

Location:

```
apps/notifications/services/email_sender.py
```

Purpose: abstracts email provider.

Contract:

```
EmailSenderService.send(  
    to: str,  
    subject: str,  
    template_name: str,  
    context: dict,  
    idempotency_key: str,  
    user: User | None = None,  
    batch: EmailBatch | None = None,  
    email_type: str = "system"  
) -> EmailEvent
```

Flow: check idempotency_key, create EmailEvent queued, render template, send via provider, update EmailEvent status.

If idempotency_key already exists, return existing EmailEvent.

13.2 EmailPreferenceService

Location:

apps/notifications/services/preferences.py

Contracts:

```
EmailPreferenceService.update_preferences(user: User, cleaned_data: dict) -> EmailPreference
EmailPreferenceService.unsubscribe(token: str) -> ServiceResult
```

Side effects: updates EmailPreference, EmailUnsubscribeToken.used_at, and ConsentRecord if digest opt-in.

13.3 WeeklyDigestService

Location:

apps/notifications/services/weekly_digest.py

Contract:

```
WeeklyDigestService.send_weekly_digest(period_start, period_end) -> EmailBatch
```

Eligibility: verified email, weekly_digest_enabled=true, active within last 14 days, usable profile, new relevant recommendations.

Idempotency key:

```
weekly_digest:{user_id}:{year}-W{week}
```

14. Privacy Services

14.1 ConsentService

Location:

apps/privacy/services/consent.py

Contract:

```
ConsentService.record(
    user: User,
    consent_type: str,
    consent_text: str,
    consent_version: str,
    request_meta: dict | None = None
) -> ConsentRecord
```

Consent types: cv_processing, email_digest, terms, privacy_policy, cookie_session_notice.

14.2 AccountDeletionService

Location:

apps/privacy/services/account_deletion.py

Contracts:

```
AccountDeletionService.request_deletion(user: User) -> DeletionRequest
AccountDeletionService.process_request(deletion_request: DeletionRequest) -> DeletionRequest
```

Flow:

1. Mark request processing.

2. Disable user login.
3. Delete CV files.
4. Delete CV parsed data.
5. Delete candidate profile.
6. Delete profile skills.
7. Delete saved jobs.
8. Delete match results.
9. Delete recommendations.
10. Delete email preferences.
11. Delete social accounts.
12. Anonymize or delete user events.
13. Delete/anonymize user.
14. Mark request completed.

14.3 PrivacyExportService

Not required in MVP.

Future contract:

```
PrivacyExportService.export_user_data(user: User) -> dict
```

15. Analytics Services

15.1 UserEventService

Location:

```
apps/analytics/services/events.py
```

Purpose: records lightweight product analytics.

Contract:

```
UserEventService.record(  
    event_type: str,  
    user: User | None = None,  
    session_key: str | None = None,  
    metadata: dict | None = None  
) -> UserEvent
```

Event types: job_search, job_detail_view, quick_match_started, quick_match_completed, signup_started, signup_completed, cv_uploaded, cv_parse_failed, profile_completed, match_generated, recommendation_clicked, saved_job_created, source_apply_clicked, email_digest_clicked, account_deleted.

Analytics failure must not break user flow.

16. Health and System Services

16.1 HealthCheckService

Location:

```
apps/core/services/health.py
```

Contract:

```
HealthCheckService.check() -> dict
```

Success output:

```
{
  "status": "ok",
  "database": "ok",
  "redis": "ok"
}
```

16.2 SystemSettingService

Location:

```
apps/core/services/settings.py
```

Contract:

```
SystemSettingService.get(key: str, default=None)
```

Examples: job_stale_after_hours, job_removed_after_hours, active_user_days, max_cv_upload_size_mb, max_ilm_calls_per_user_day.

If setting is missing, return default.

17. Celery Task Contracts

Celery tasks should be thin wrappers around services.

Bad:

```
@shared_task
def parse_cv(cv_id):
    # 200 lines of parsing logic
```

Good:

```
@shared_task
def parse_cv(cv_id):
    cv = CVUpload.all_objects.get(id=cv_id)
    return CVParsingService.parse(cv)
```

Ingestion tasks

```
sync_france_travail_jobs()
normalize_raw_job_record(raw_record_id)
extract_job_skills(job_id)
mark_stale_and_expired_jobs()
```

CV tasks

```
parse_cv(cv_upload_id)
```

Must use CVUpload.all_objects, but abort if deleted_at is not null.

Matching tasks

```
generate_match_explanation(match_result_id)
```

Recommendation tasks

```
refresh_user_recommendations(user_id, trigger_type)
refresh_active_users_recommendations()
```

Email tasks

`send_weekly_digest(period_start, period_end)`

Privacy tasks

`process_account_deletion(deletion_request_id)`

18. View-to-Service Map

Public Job Search

Route:

`/jobs/`

Calls: `JobSearchService.search()`, `UserEventService.record(job_search)`.

Job Detail

Route:

`/jobs/<uuid:public_id>/`

Calls: `JobQueryService.get_public_job()`, `JobRevalidationService.revalidate_if_needed()`, `UserEventService.record(job_detail_view)`.

Quick Match

Route:

`/jobs/<uuid:public_id>/quick-match/`

Calls: `JobQueryService.get_public_job()`, `QuickMatchService.run_quick_match()`.

CV Upload

Route:

`/dashboard/cv/`

Calls: `ConsentService.record()`, `CVUploadService.upload_cv()`.

Profile Edit

Route:

`/dashboard/profile/`

Calls: `ProfileUpdateService.update_profile()`.

Full Match

Route:

`/jobs/<uuid:public_id>/match/`

Calls: `JobQueryService.get_public_job()`, `MatchResultService.create_match_result()`.

Recommendations Dashboard

Route:

`/dashboard/recommendations/`

Calls: `RecommendationQueryService.get_dashboard_recommendations()`.

Save Job

Route:

`/jobs/<uuid:public_id>/save/`

Calls: `SavedJobService.save_job()`.

Email Preferences

Route:

`/dashboard/email-preferences/`

Calls: `EmailPreferenceService.update_preferences()`.

Delete Account

Route:

`/dashboard/settings/delete-account/`

Calls: `AccountDeletionService.request_deletion()`.

19. Admin Action Service Map

Admin actions should call services, not duplicate logic.

Examples:

```
Re-normalize selected raw records -> JobNormalizationService.normalize()
Re-extract job skills -> JobSkillExtractionService.extract_for_job()
Map unmatched skill -> UnmatchedSkillReviewService.map_candidate()
Refresh user recommendations -> RecommendationService.refresh_for_user(..., manual_admin)
Re-parse CV -> CVParsingService.parse(), abort if CV is deleted
```

20. Service Dependency Rules

Allowed dependencies:

```
jobs services can call skills services
cvs services can call skills and llm services
matching services can read profiles, jobs, skills
recommendations services can call matching services
notifications services can call recommendations query services
```

privacy services can call cvs/recommendations/matching cleanup services
analytics should be independent

Forbidden dependencies:

skills services should not call matching
skills services should not call recommendations
jobs normalization should not call recommendations directly
views should not call OpenRouterClient directly
templates should not contain scoring logic
models should not call external APIs

21. Transaction and Locking Rules

Recommendation Refresh

Use transaction when replacing active recommendations:

```
transaction:  
  mark old recommendations stale  
  upsert new recommendations  
  complete RecommendationRun
```

CV Upload

Use transaction for metadata operations. File storage itself is outside DB transaction, so handle cleanup if DB save fails.

Email Sending

Do not send email inside long database transaction.

Correct flow:

```
create EmailEvent queued  
commit  
send email  
update EmailEvent sent/failed
```

Ingestion

Do not wrap entire ingestion run in one huge transaction. Use smaller transactions per job or per page.

22. Service Acceptance Criteria

Service Contracts v1 is accepted when:

Each main business domain has a service boundary.
External API calls are isolated.
LLM calls are isolated.
Views remain thin.
Celery tasks remain thin.
Matching score has one authority.
Recommendations reuse matching service.
Job ingestion preserves raw payloads.
CV parsing uses deterministic + LLM extraction.
Email sends are idempotent.
Privacy deletion has a clear owner.
Admin actions reuse services.

23. Final Service Design Decision

TuniTech Abroad should use a service-layer architecture inside the Django monolith.

Final implementation rule:

- No business logic directly in views.
- No external API calls in models.
- No LLM calls in views.
- No scoring logic in templates.
- No duplicated recommendation logic in tasks.

The service layer is what keeps the project maintainable as it grows.

24. Next Document

After Service Contracts v1 validation, the next planning document is Data Flow / Sequence Diagrams v1.

It should define step-by-step flows for:

- Signup/onboarding
- France Travail ingestion
- Job normalization
- Skill extraction
- CV upload/parsing
- Anonymous quick match
- Full authenticated match
- Recommendation refresh
- Weekly digest
- Account deletion